

ECC Skills for AI Assisted Development

A practical reference for Claude Code Codex Cursor and other coding agents

Everything Claude Code (ECC) is an open-source engineering toolbox for AI coding agents. It supplies reusable workflows for planning, implementation, testing, security, review and verification. The central idea is simple: install a reviewed engineering process once, then apply the relevant parts consistently in each project.

1 What an ECC skill is

An ECC skill is a reusable instruction set stored in a SKILL.md file. Its header identifies the skill and when it applies; its body defines the workflow, constraints and completion checks. Examples include springboot-patterns, springboot-tdd, springboot-security and springboot-verification. ECC also contains specialist agents, commands, rules, hooks and memory tools. Installing ECC makes these components discoverable, but it does not run every skill automatically.

2 How ECC works

- The user gives the agent a concrete outcome and names important skills when the method matters.
- The agent reads repository instructions, the active specification and only the relevant skill guidance.
- SDD turns requirements into acceptance criteria, an approved plan and small implementation tasks.
- TDD repeats RED, GREEN and REFACTOR for one observable behavior at a time.
- Stack-specific skills guide architecture, security and testing without replacing the approved contract.
- Verification runs focused checks and the repository's complete build before completion is reported.

3 Installation by coding agent

Use an official source and choose one installation method per agent. Do not combine a native plugin install with a full manual install in the same agent.

Agent	Installation
Claude Code	<code>npx ecc-universal@2.2.1 setup</code> , or inside Claude Code: <code>/plugin marketplace add https://github.com/affaan-m/ECC</code> followed by <code>/plugin install ecc@ecc</code> . Use only one path.
Codex	<code>codex plugin marketplace add affaan-m/ECC</code> ; <code>codex plugin add ecc@ecc</code> ; verify with <code>codex plugin list --json</code> . Review native hooks before trusting them.
Cursor	Clone <code>affaan-m/ECC</code> , enter the repository, then run <code>./install.sh --profile minimal --target cursor</code> .
Gemini CLI	From a reviewed ECC clone, run <code>./install.sh --profile minimal --target gemini</code> .
OpenCode	From a reviewed clone: <code>npm install</code> ; <code>npm run build:opencode</code> ; <code>./install.sh --profile full --target opencode --enable-hooks</code> .

4 How to use an ECC skill

After installation, open the target repository and ask for both the outcome and the workflow. A useful prompt is:

```
Use ECC's springboot-patterns, springboot-tdd and springboot-verification skills.  
Implement ticket claiming from the approved specification. Write and run a failing  
test first, make the minimum change, and run the full Maven verification before  
reporting completion.
```

Skills describe the reusable method; they should not become product requirements. Stable business rules belong in specifications, permanent repository rules belong in AGENTS.md, and feature plans belong under docs/plans. This separation keeps SDD uniform across features and commits.

5 Example from ResolveHub Lite

ResolveHub Lite is a Java 21 Spring Boot support-ticket API backed by PostgreSQL, Flyway and Testcontainers. The project shows how ECC-style skills can turn a broad exercise into traceable engineering work.

SDD. The problem was divided into platform foundation, identity and access, ticket intake, claiming, resolution, summary reporting and operability. Each capability received a specification with stable requirement and acceptance-criterion IDs. Approved work was then mapped into an implementation plan and small tasks. AGENTS.md makes those specifications authoritative so a later skill update cannot silently change the product contract.

TDD. Each behavior began with a failing test. Pagination and sorting were introduced through a policy test that failed before the production policy existed. A Unicode validation defect was reproduced with boundary and API tests before the implementation was aligned with PostgreSQL character counting. Database-backed concurrency tests proved that two agents claiming one ticket produce one winner and one conflict, instead of relying on JVM synchronization.

ECC contribution. Java and Spring Boot pattern skills guided structure; the security skill supported authentication and CSRF review; verification and delivery skills checked focused tests, the full Maven build and completion claims. Project-local SDD and TDD skills remained authoritative.

6 Important lesson from the POC

The POC began with copied project-local SDD and TDD skill files. The native ECC plugin was installed later, so it strengthened later implementation, defect correction, review and verification. It should not be described as natively installed from the first commit. For a new project, install ECC first, select only the needed skills, define instruction precedence, approve the specification and then begin the first TDD cycle.

7 Practical recommendations

- Start with a minimal profile and add only skills relevant to the technology and task.
- Name critical skills explicitly in the prompt; installation alone does not guarantee their use.
- Keep specifications stable and change them deliberately when behavior changes.
- Require observable tests and the full repository verification command before accepting completion.
- Use official sources, pin releases when reproducibility matters and review hooks before trusting them.

Official source: github.com/affaan-m/ECC | Checked against ECC 2.2.1